

# Informe Proyecto Final: My Letter Locker

## Estudiantes:

Brahian Stiven Mosquera Valanta

Jorge Iván Acosta Aristizábal



**Universidad del Quindío**

**Facultad Ingeniería**

**Ingeniería de Sistemas y Computación**

Teoría de Lenguajes Formales

Armenia, Quindío

Mayo de 2026

# Índice

|                                                         |          |
|---------------------------------------------------------|----------|
| <b>Introducción</b>                                     | <b>3</b> |
| <b>1. Definición y requisitos del proyecto</b>          | <b>3</b> |
| 1.1. Requisitos Técnicos . . . . .                      | 4        |
| 1.2. Requisitos Funcionales . . . . .                   | 4        |
| <b>2. Diseño e implementación</b>                       | <b>4</b> |
| 2.1. Patrones a identificar . . . . .                   | 5        |
| 2.1.1. Comandos de formato y configuración . . . . .    | 5        |
| 2.1.2. Datos estructurados . . . . .                    | 6        |
| 2.2. Tecnologías y herramientas seleccionadas . . . . . | 7        |
| 2.3. Estructura del proyecto . . . . .                  | 7        |
| 2.4. Componentes principales . . . . .                  | 7        |
| 2.4.1. Clase Automaton . . . . .                        | 7        |
| 2.4.2. LetterLockerPreviewer . . . . .                  | 8        |
| 2.5. Algoritmo de cifrado . . . . .                     | 8        |
| 2.6. Diseño de interfaz e interacción . . . . .         | 9        |
| <b>3. Evidencias</b>                                    | <b>9</b> |
| <b>4. Conclusiones</b>                                  | <b>9</b> |

La Teoría de Lenguajes Formales proporciona los fundamentos matemáticos para el procesamiento automático de texto y el análisis sintáctico. Entre sus herramientas principales se encuentran los autómatas finitos, modelos abstractos compuestos por estados y transiciones que procesan secuencias de entrada símbolo a símbolo. Estos formalismos se aplican tanto al reconocimiento de datos estructurados (correos electrónicos, URLs, fechas) como al análisis de comandos de personalización. El presente proyecto, desarrollado como trabajo final de la asignatura Teoría de Lenguajes Formales de la Universidad del Quindío, aplica estos conceptos al desarrollo de *My Letter Locker*, una aplicación web para compartir cartas digitales de forma privada.

## **1. Definición y requisitos del proyecto**

*My Letter Locker* es una aplicación web que permite compartir cartas digitales de forma privada, sin necesidad de registro ni almacenamiento en servidor. El contenido de cada carta se cifra mediante un algoritmo de cifrado simétrico y se embebe directamente en la URL, de manera que solo quien posea el enlace y la palabra secreta puede descifrar su contenido.

Para procesar las entradas del usuario, la aplicación emplea autómatas finitos como mecanismo formal de análisis. Estos modelos reconocen tanto datos estructurados (correos electrónicos, URLs, fechas) como comandos de personalización que modifican la apariencia del texto y el entorno visual.

En las siguientes tablas se detallan los requisitos técnicos y funcionales del sistema.

## 1.1. Requisitos Técnicos

| Requisito                       | Descripción                                                                                                 |
|---------------------------------|-------------------------------------------------------------------------------------------------------------|
| validación de patrones de datos | Verificación de formato mediante autómatas finitos para campos como correo electrónico, URL, fecha y título |
| Reconocimiento de comandos      | Análisis sintáctico de comandos de formato y carta mediante autómatas finitos                               |
| Generación de identificadores   | Creación de tokens únicos embebidos en las URLs                                                             |
| Cifrado simétrico               | Algoritmo para cifrar y descifrar el contenido de la carta                                                  |

## 1.2. Requisitos Funcionales

| Requisito                      | Descripción                                                                                                  |
|--------------------------------|--------------------------------------------------------------------------------------------------------------|
| Creación sin registro          | El usuario redacta el mensaje y define la palabra secreta sin necesidad de cuenta                            |
| Cifrado embebido en URL        | El contenido se cifra y embebe directamente en la URL, sin almacenamiento en servidor                        |
| Comandos de carta              | Comandos globales para configurar la palabra secreta, tema visual, tipografía, iluminación y música de fondo |
| Comandos de formato            | Comandos inline para aplicar negrita, cursiva, títulos y otros estilos al texto de la carta                  |
| Desbloqueo con palabra secreta | El destinatario debe ingresar la palabra secreta para descifrar el mensaje                                   |

## 2. Diseño e implementación

El desarrollo de *My Letter Locker* se fundamenta en una arquitectura modular que separa claramente los componentes de procesamiento de texto, gestión de comandos,

cifrado y visualización. Esta sección describe los patrones reconocidos mediante autómatas finitos, las tecnologías utilizadas y la estructura del proyecto.

## 2.1. Patrones a identificar

La aplicación reconoce dos categorías de patrones mediante autómatas finitos implementados en TypeScript:

### 2.1.1. Comandos de formato y configuración

Los comandos siguen una sintaxis declarativa con delimitadores de dólar (\$). Un comando tiene la estructura `$nombre$valor$` o `$nombre:modificador$valor$`, donde:

- `nombre` identifica el tipo de comando.
- `modificador` (opcional) especifica parámetros adicionales.
- `valor` contiene el contenido al que se aplica el comando.

Los comandos reconocidos incluyen:

- `$bold$texto$` — Aplica negrita al texto.
- `$italic$texto$` — Aplica cursiva al texto.
- `$title:n$texto$` — Renderiza texto como encabezado de nivel  $n$  (1 a 6).
- `$pass$palabra$` — Define la palabra secreta para el cifrado.

El autómata que reconoce estos patrones utiliza tres estados:

- **Q0 (Texto plano):** Estado inicial. Copia los caracteres al resultado hasta encontrar el primer \$.

- **Q1 (Nombre del comando):** Acumula caracteres del nombre del comando hasta el siguiente \$.
- **Q2 (Valor del comando):** Acumula el valor hasta encontrar el delimitador final \$, momento en que ejecuta el comando correspondiente.

La Figura 1 ilustra el diagrama de estados del autómata reconocedor.

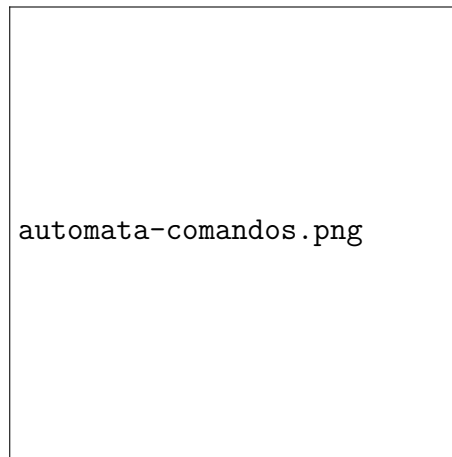


Figura 1: Diagrama de estados del autómata reconocedor de comandos

### 2.1.2. Datos estructurados

Para validar campos de entrada se emplean autómatas finitos que verifican el formato de:

- **Correos electrónicos:** Formato `usuario@dominio.ext`.
- **URLs:** Protocolo opcional seguido de dominio y ruta.
- **Fechas:** Formato ISO 8601 (AAAA-MM-DD).
- **Títulos:** Cadenas no vacías con longitud máxima de 100 caracteres.

## 2.2. Tecnologías y herramientas seleccionadas

La pila tecnológica del proyecto prioriza la simplicidad y la portabilidad:

| Tecnología     | Uso en el proyecto                                      |
|----------------|---------------------------------------------------------|
| TypeScript     | Lenguaje principal, tipado estático para mayor robustez |
| Vite           | Bundler y servidor de desarrollo                        |
| Web Crypto API | Cifrado AES-GCM y derivación de claves PBKDF2           |
| Pako           | Biblioteca de compresión gzip para JavaScript           |
| HTML5/CSS3     | Interfaz de usuario                                     |

Se optó por Web Crypto API nativa del navegador en lugar de bibliotecas externas de criptografía, lo que elimina dependencias y reduce el tamaño del bundle.

## 2.3. Estructura del proyecto

El código fuente se organiza en módulos funcionales dentro del directorio `src/`:

- `automaton/` — Implementación del autómata genérico y el reconocedor de comandos.
- `commands/` — Definición y registro de comandos de formato.
- `encryption/` — Servicio de cifrado y descifrado.
- `routing/` — Sistema de enrutamiento de vistas.
- `views/` — Componentes de interfaz (home, reader, readcard).

## 2.4. Componentes principales

### 2.4.1. Clase Automaton

La clase `Automaton` proporciona la lógica base para el reconocimiento de patrones. Sus componentes principales son:

- **Estados:** Representados mediante `StatusDeclaration`.

- **Transiciones:** Definidas por `TransitionDeclaration` con símbolo (string o expresión regular) y estado destino.
- **Ejecución:** El método `run(cadena)` procesa la entrada símbolo a símbolo, lanzando error si no existe transición válida.
- **Callbacks:** Permiten ejecutar acciones al atravesar una transición (`onTransition`).

#### 2.4.2. LetterLockerPreviewer

Esta clase coordina el autómata con el `CommandManager`:

1. El autómata procesa el texto carácter a carácter.
2. Al encontrar un comando completo (estado  $Q2 \rightarrow Q0$ ), invoca el comando correspondiente.
3. El resultado HTML se concatena al resultado final.
4. Si se detecta `$pass$`, se extrae y almacena la palabra secreta.

### 2.5. Algoritmo de cifrado y compresión

El sistema implementa un flujo de procesamiento que combina compresión y cifrado para permitir cartas de hasta 60 000 caracteres:

1. **Compresión:** El texto plano se comprime mediante el algoritmo gzip utilizando la biblioteca Pako. Esta compresión reduce significativamente el tamaño del contenido antes del cifrado.
2. **Derivación de clave:** PBKDF2 con 100 000 iteraciones y SHA-256 deriva una clave AES de 256 bits a partir de la palabra secreta.



3. **Cifrado:** AES-GCM con vector de inicialización aleatorio proporciona cifrado autenticado.
4. **Formato de salida:** Base64url de [salt (16 bytes) | iv (12 bytes) | ciphertext].

El flujo inverso (descifrado) invierte estos pasos: descifrar, descomprimir y procesar los comandos de formato.

El formato Base64url (sin padding y con caracteres URL-safe) permite embeber el texto cifrado directamente en la URL.

## 2.6. Diseño de interfaz e interacción

La aplicación presenta tres vistas principales:

- **Home (/):** Formulario de creación de carta. Incluye editor de texto con soporte para comandos, campo de palabra secreta, y botón para generar el enlace cifrado.
- **Reader (/read):** Página de lectura. Solicita la palabra secreta al usuario y, si es correcta, muestra el contenido descifrado.
- **ReadCard (/read/card):** Vista final que renderiza la carta con formato HTML.

La navegación entre vistas se gestiona mediante un sistema de enrutamiento basado en el hash de la URL, manteniendo la arquitectura de aplicación de página única (SPA).

## 3. Evidencias

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi.

Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

## **4. Conclusiones**

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec odio elit, dictum in, hendrerit sit amet, egestas sed, leo. Praesent feugiat sapien aliquet odio. Integer vitae justo. Aliquam vestibulum fringilla lorem. Sed neque lectus, consectetur at, consectetur sed, eleifend ac, lectus. Nulla facilisi. Pellentesque eget lectus. Proin eu metus. Sed porttitor. In hac habitasse platea dictumst. Suspendisse eu lectus. Ut mi mi, lacinia sit amet, placerat et, mollis vitae, dui. Sed ante tellus, tristique ut, iaculis eu, malesuada ac, dui. Mauris nibh leo, facilisis non, adipiscing quis, ultrices a, dui.